

Snowflake Mastery — PJ's Academy

The complete Snowflake course: from zero to SnowPro certified, with 20 hands-on projects.

Snowflake is the world's leading cloud data platform. Snowflake engineers in India earn **₹18–45 LPA**, and the SnowPro certifications are among the fastest-growing cloud credentials.

This course takes you from "what is a data warehouse?" to building production data platforms and passing **SnowPro Core + Advanced** certifications.

Who This Is For

- Data analysts wanting to level up to data engineering
- Python/SQL developers moving into the modern data stack
- Data scientists who need to work with cloud warehouses
- Anyone targeting SnowPro certification + a ₹18L+ data role

Prerequisites: Basic SQL (SELECT, JOIN, GROUP BY). We teach the rest.

Course Structure — 8 Modules

Module	Topic	Outcome
1	Snowflake Architecture & Fundamentals	Understand why Snowflake is different
2	Loading & Unloading Data	Get data in and out at scale
3	Advanced SQL & Performance	Write fast, cost-efficient queries
4	Time Travel, Cloning & Data Sharing	Master Snowflake's superpowers
5	Streams, Tasks & Snowpipe	Build real-time pipelines
6	Snowpark (Python/Java/Scala)	Run code inside Snowflake
7	Security, Governance & RBAC	Enterprise-grade data security
8	Cost Optimisation & Best Practices	Run Snowflake without burning money

Module 1 — Architecture & Fundamentals

The 3-layer architecture that makes Snowflake special:

CLOUD SERVICES LAYER

(authentication, query optimisation)

← Brain: auth, optimiser, metadata

COMPUTE LAYER (Virtual Warehouses)	← Muscle: independent, scalable
[WH1] [WH2] [WH3] ← scale independently	
STORAGE LAYER (Cloud Object Storage)	← Memory: cheap, infinite, shared
(micro-partitions, columnar, compressed)	

Why this matters: Compute and storage are **separated**. You can spin up 10 warehouses querying the same data with zero contention, scale a warehouse up for a heavy job then down, and pay only for what you use per second.

Key concepts covered: - Virtual warehouses (XS → 6XL) and multi-cluster warehouses - Micro-partitions and automatic clustering - Result cache, metadata cache, and warehouse cache (3 cache layers) - Editions: Standard, Enterprise, Business Critical

```
-- Your first warehouse
CREATE WAREHOUSE learning_wh
  WAREHOUSE_SIZE = 'XSMALL'
  AUTO_SUSPEND = 60      -- suspend after 60s idle (save money!)
  AUTO_RESUME = TRUE
  INITIALLY_SUSPENDED = TRUE;

-- Create a database and schema
CREATE DATABASE pjs_academy;
CREATE SCHEMA pjs_academy.sales;

-- Snowflake scales instantly - resize mid-workload
ALTER WAREHOUSE learning_wh SET WAREHOUSE_SIZE = 'LARGE';
```

Module 2 — Loading & Unloading Data

Covered: - Stages (internal, external S3/Azure/GCS, user, table) - `COPY INTO` — the workhorse loader - File formats (CSV, JSON, Parquet, Avro, ORC) - Bulk loading vs Snowpipe (continuous) - Semi-structured data with the `VARIANT` type

```
-- Create a file format
CREATE FILE FORMAT csv_format
  TYPE = 'CSV'
  FIELD_DELIMITER = ','
  SKIP_HEADER = 1
  NULL_IF = ('NULL', 'null', '');

-- Create an external stage pointing to S3
CREATE STAGE sales_stage
  URL = 's3://my-bucket/sales/'
  FILE_FORMAT = csv_format;

-- Load data - Snowflake parallelises automatically
COPY INTO sales_data
  FROM @sales_stage
  ON_ERROR = 'CONTINUE'
  PURGE = TRUE;

-- Query JSON like it's structured (VARIANT magic)
SELECT
  raw:customer.name::STRING AS customer_name,
  raw:order.total::NUMBER AS order_total,
  raw:items[0].sku::STRING AS first_item
FROM orders_json;
```

⚡ Module 3 — Advanced SQL & Performance

Covered: - Window functions, QUALIFY, LATERAL FLATTEN - Query profiling and the query profile UI - Clustering keys and when to use them - Search optimisation service - Materialised views vs regular views

```
-- QUALIFY - filter window functions without a subquery (Snowflake gem)
SELECT customer_id, order_date, amount,
       ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY amount DESC) AS rn
FROM orders
QUALIFY rn <= 3; -- top 3 orders per customer, one clean query

-- FLATTEN semi-structured arrays into rows
SELECT o.order_id, item.value:sku::STRING AS sku
FROM orders o,
LATERAL FLATTEN(input => o.raw:items) item;
```

🕒 Module 4 — Time Travel, Cloning & Data Sharing

Snowflake's three superpowers:

```
-- TIME TRAVEL - query data as it was in the past
SELECT * FROM orders AT(OFFSET => -3600); -- 1 hour ago
SELECT * FROM orders BEFORE(STATEMENT => 'query_id'); -- before a mistake

-- Undrop a table you accidentally deleted!
UNDROP TABLE orders;

-- ZERO-COPY CLONING - clone a whole database instantly, no storage cost
CREATE DATABASE prod_clone CLONE production_db;
-- (only stores the DIFFERENCES as you change the clone)

-- SECURE DATA SHARING - share live data, no copying
CREATE SHARE sales_share;
GRANT SELECT ON TABLE sales_data TO SHARE sales_share;
-- Partner queries YOUR data live, you both save storage
```

Why interviewers love this: Zero-copy cloning for instant dev/test environments and Time Travel for disaster recovery are uniquely Snowflake.

🔄 Module 5 — Streams, Tasks & Snowpipe

Build real-time pipelines with Snowflake-native CDC:

```
-- STREAM - track changes (inserts/updates/deletes) on a table
CREATE STREAM orders_stream ON TABLE orders;

-- TASK - scheduled or triggered SQL execution
CREATE TASK process_new_orders
  WAREHOUSE = etl_wh
  SCHEDULE = '1 MINUTE'
  WHEN SYSTEM$STREAM_HAS_DATA('orders_stream')
  AS
```

```

INSERT INTO orders_summary
SELECT order_date, SUM(amount)
FROM orders_stream
GROUP BY order_date;

-- SNOWPIPE - auto-load files the moment they land in S3
CREATE PIPE sales_pipe
  AUTO_INGEST = TRUE
AS
  COPY INTO sales_data FROM @sales_stage;

```

This is the modern data stack — near-real-time ELT without external tools.

Module 6 — Snowpark

Run Python/Java/Scala INSIDE Snowflake — no data movement:

```

from snowflake.snowpark import Session
from snowflake.snowpark.functions import col, avg, when

# Connect
session = Session.builder.configs(connection_params).create()

# DataFrame API - looks like pandas, runs on Snowflake compute
df = session.table("sales_data")
result = (df
    .filter(col("amount") > 1000)
    .group_by("region")
    .agg(avg("amount").alias("avg_sale"))
    .sort(col("avg_sale").desc()))

result.show()

# Deploy a Python UDF that runs in Snowflake
@udf(name="predict_churn", is_permanent=True, stage_location="@udf_stage")
def predict_churn(tenure: int, charges: float) -> float:
    # Your ML model runs INSIDE Snowflake
    return model.predict([[tenure, charges]])[0]

```

Why it's huge: Train and serve ML models where the data lives — no exporting to external systems.

Module 7 — Security, Governance & RBAC

Covered: - Role-Based Access Control (RBAC) hierarchy - Row-level security & column-level masking - Dynamic data masking - Object tagging & data classification - Network policies & MFA

```

-- RBAC hierarchy
CREATE ROLE analyst;
CREATE ROLE data_engineer;
GRANT ROLE analyst TO ROLE data_engineer; -- engineers inherit analyst rights

-- Dynamic data masking - hide PII from unauthorised roles
CREATE MASKING POLICY email_mask AS (val STRING) RETURNS STRING ->
CASE
  WHEN CURRENT_ROLE() IN ('ADMIN') THEN val

```

```
ELSE REGEXP_REPLACE(val, '.*@', '****@') -- mask for everyone else
END;
```

```
ALTER TABLE customers MODIFY COLUMN email
SET MASKING POLICY email_mask;
```

```
-- Row-level security
```

```
CREATE ROW ACCESS POLICY region_policy AS (region STRING) RETURNS BOOLEAN ->
region = CURRENT_REGION() OR CURRENT_ROLE() = 'ADMIN';
```

Module 8 — Cost Optimisation

The skill that gets you promoted — running Snowflake efficiently:

- Right-sizing warehouses (bigger isn't always better)
- Auto-suspend & auto-resume tuning
- Resource monitors to cap spend
- Query optimisation to reduce compute
- Storage optimisation (Time Travel retention, transient tables)

```
-- Resource monitor - hard stop at budget limit
CREATE RESOURCE MONITOR monthly_budget
WITH CREDIT_QUOTA = 100
TRIGGERS
  ON 80 PERCENT DO NOTIFY
  ON 100 PERCENT DO SUSPEND;

ALTER WAREHOUSE etl_wh SET RESOURCE_MONITOR = monthly_budget;

-- Find your most expensive queries
SELECT query_text, warehouse_name,
       execution_time/1000 AS seconds,
       credits_used_cloud_services
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
ORDER BY execution_time DESC
LIMIT 20;
```

Certifications This Course Prepares You For

Certification	Level	What It Proves
SnowPro Core	Foundational	You can use Snowflake proficiently
SnowPro Advanced: Data Engineer	Advanced	You build production pipelines
SnowPro Advanced: Data Scientist	Advanced	You do ML on Snowflake
SnowPro Advanced: Architect	Expert	You design Snowflake platforms

The course includes **200+ practice questions**, **4 full mock exams**, and hands-on labs matching every exam domain.



20 Hands-On Projects

See the [Snowflake Projects Vault](#) — 20 advanced projects from beginner pipelines to real-time streaming platforms and ML-in-warehouse systems.



Enrol

-  pjsacademy.com
 -  hello@pjsacademy.com
 -  [@pjsacademy.datascience](https://twitter.com/pjsacademy.datascience)
-

Course:  *Snowflake Mastery — PJ's Academy*

 Snowflake Mastery — PJ's Academy · pjsacademy.com · hello@pjsacademy.com · [@pjsacademy.datascience](https://twitter.com/pjsacademy.datascience)